# English Symposium

By: Taylor Sloan

# Why are you here?

# Why am I here?

```
                    requireActivity().runOnUiThread(() -> {
                        populateBeerList(context);
                    });
                } catch (Exception e) {
                    Log.d( tag: "DUBREW_ERROR", e.toString());
                }
            } else if (response.code() == 401){
                utils.toSplitter(requireActivity());
            }
        }
    });
}
} catch (Exception e){
    Log.d( tag: "DUBREW_ERROR", e.toString());
}
}

1 usage
public View.OnClickListener beerClicked(int index) {
    return new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            // Create a bundle with the index
            Bundle args = new Bundle();
            args.putInt("index", index);

            // Navigate using NavController
            NavHostFragment.findNavController( fragment: BeersFragment.this)
                    .navigate(R.id.action_beersFragment_to_beerPageFragment, args);
        }
    };
}

1 usage
public View.OnClickListener reviewClicked(int index) {
    return new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            // Create a bundle with the index
            Bundle args = new Bundle();
```

# Who am I?

+

```
                    populateBeerList(context);
                });
                } catch (Exception e) {
                    Log.d(tag: "DUBREN_ERROR", e.toString());
                }
                } else if (response.code() == 401){
                    utils.toSplitter(requireActivity());
                }
            }
        });
    }
} catch (Exception e){
    Log.d(tag: "DUBREN_ERROR", e.toString());
}
}

1 usage
public View.OnClickListener beerClicked(int index) {
    return new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            // Create a bundle with the index
            Bundle args = new Bundle();
            args.putInt("index", index);

            // Navigate using NavController
            NavHostFragment.findNavController(fragment: BeersFragment.this)
                    .navigate(R.id.action_beersFragment_to_beerPageFragment, args);
        }
    };
}

1 usage
public View.OnClickListener reviewClicked(int index) {
    return new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            // Create a bundle with the index
            Bundle args = new Bundle();
```

=

?

# '''Coding Lessons: Comment'''

```
# Text within code that is solely for a human reader’s benefit
# It is ignored by the computer, just increasing readability

# Ex:
    '''Comment'''
    /* Comment */
    # Comment
    // Comment
```

# '''Coding Lessons: Variables'''

```
# Variables represent other values

# For instance, your name is a variable for you as a person

# Ex:
    x = 5
    var x:number = 5;
    int x = 5;
```

# '''Coding Lessons: Data Types'''

| Type | What is it? | Example |
| --- | --- | --- |
| Integer (int) | A whole number | `5, 99, -1` |
| Float | A number with a decimal place | `5.2, 9.0, -1.5` |
| String (`str`) | Anything used to represent text. May have letters, numbers, characters, etc. Typically enclosed by quotes. | `"a", "Hello", "78", "10395hklgsdfr", "😍"` |
| List (lst) | A collection of multiple objects. A list might have 5 strings in it, 3 numbers, 2 other lists, etc. Lists are wrapped by brackets. They may even be nested. | `[0, 1, 2, 3, 4, 5],`<br>`["a", "b", "c"],`<br>`[1, "a", "91", 12],`<br>`[1, [2, 3], 4]` |

# '''Coding Lessons: Loops'''

# Sections of code that will loop and repeat until pre-established conditions are met

```
# While Loop
number = 0
    while (number < 5):
        print(number)
        number = number + 1
```

```
# For loop
for i in range(0,6):
        print(i)
```

# '''Coding Lessons: Functions'''

```
# Allow sections of code to be called and recalled from other locations

# Increases readability and functionality

# Can pass and return values


function driveToWork(){
     drive(5);
     left();
     drive(2);
}
driveToWork()
```

# '''Coding Lessons: Boolean Operators'''

| Operator | Representation | Example | Value |
|---|---|---|---|
| Not | `not`, `!` | `!true`<br>`not False` | `false`<br>`True` |
| And | `and`, `&&`, `&` | `true && true`<br>`True and False`<br>`false & false` | `true`<br>`False`<br>`false` |
| Or | `or`, `\|\|`, `\|` | `true \|\| true`<br>`True or False`<br>`false \| false` | `true`<br>`True`<br>`false` |

# '''Coding Lessons: How to read code'''

# May be non-linear due to functions or other elements

# Find an entry point, then follow it

# Typically there is a function named “main” or something else that pops out

# Look for what is called outside of a function: i.e. driveToWork() example

```
'''Poem 1'''


random.seed("Determinism")
choices=["Yes", "No"]
print("Is there free will?")
print(choices[random.randint(0,2)])
```

# '''Poem 2'''

```
import time
def call(filename):
     with open(filename, 'r') as file:
     print(file.read())
def born():
     while True:
     call("family/grandad.txt")
     time.sleep(31556952)
born()
```

# '''Poem 2'''

```
[tsloan@tsloan code_poems]$ python3 loss.py
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Success
Traceback (most recent call last):
    File "loss.py", line 12, in <module>
        born()
        ~~~~^^
    File "loss.py", line 9, in born
        call("family/grandad.txt")
        ~~~~^^^^^^^^^^^^^^^^^^^^^^
    File "loss.py", line 4, in call
        with open(filename, 'r') as file:
             ~~~~^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'family/grandad.txt'
[tsloan@tsloan code_poems]$ python3 -c "while True: miss('family/grandad.txt')"⏎
```

# '''Poem 3: Excerpt'''

```
void readNotification(string notif, stack<string>& emotions){
	string details = click(notif);
	if (details.find("Offer Letter") != string::npos){
		emotions.push("Fear");
		emotions.push("Nervousness");
		for (size_t i = 0; i < 10; i++){
			emotions.push("Excitement");
			emotions.push("Happiness");
		}
	} else if (details.find("Soon") != string::npos){
		emotions.push("Hope");
		emotions.push("Frustration");
		emotions.push("Fear");
		emotions.push("Nervousness");
	} else if (details.find("We regret") != string::npos){
		for (size_t i = 0; i < 3; i++){
			emotions.push("Frustration");
			emotions.push("Fear");
			emotions.push("Nervousness");
			emotions.push("Annoyance");
		}
	}
}
```

# '''Reflecting on the medium'''

# Code prioritizes function; Poetry prioritizes form.

# Code lacks emotion. It is intended to be clinical and objective. There is no word choice in code

# Accessibility: How can I make it so anyone can read this?

# Attributing meaning to established coding concepts

    # Pseudorandom/Set seeds

    # Error messages

    # Stacks for emotions

# Unique and powerful when implemented well

'''If you would like to see more:'''